

PLAN DE IMPLEMENTACIÓN ENTORNO DE DESARROLLO WEB

PROYECTO: TAG-OK

Tabla de contenidos

Historial de revisiones	3
Introducción	3
Resumen Ejecutivo	4
Alcance de la implementación	4
Elementos del entorno	4
Herramientas y Tecnologías involucradas	4
Preparación del ambiente de desarrollo	5
Configuración del proyecto	5
Configuración de claves y variables de entorno	5
Verificación de implementación	5
Resolución de problemas comunes	6

Historial de revisiones

Fecha	Versión	Descripción	Autor
20 de Junio de 2026	1.0	Versión inicial del documento	Diego Rodríguez, Paulina Troncoso, Ricardo Sánchez

Introducción

El presente documento describe el procedimiento para la implementación y configuración del entorno de desarrollo del panel web administrativo del proyecto TAG OK. El objetivo es establecer una guía clara que permita a cualquier miembro del equipo preparar su estación de trabajo para ejecutar y modificar el frontend web de la plataforma.

El panel web está desarrollado con React 19 y TypeScript, utilizando Vite como herramienta de build y Leaflet para la visualización de mapas interactivos. La interfaz permite la gestión de autopistas, pódicos y tarifas, así como la visualización de rutas calculadas sobre un mapa de Santiago.

Este plan cubre desde la instalación de Node.js hasta la verificación del panel web en funcionamiento y su conexión con el backend.

Resumen Ejecutivo

El entorno de desarrollo web de TAG OK requiere Node.js como runtime y npm como gestor de paquetes. El proyecto utiliza Vite para un desarrollo rápido con recarga en caliente, React 19 con TypeScript para la interfaz de usuario, Leaflet para los mapas interactivos, TanStack React Query para la obtención de datos, y Supabase Auth para la autenticación de usuarios administradores.

La implementación se divide en tres fases: instalación de prerequisites, configuración del proyecto con las variables de entorno necesarias, y verificación de la conexión con el backend y los servicios externos.

Alcance de la implementación

Este documento abarca la instalación de Node.js, la clonación y configuración del proyecto routes-ui, la configuración de las variables de entorno para la conexión con Supabase y el backend, y la verificación del panel web en funcionamiento. Queda fuera del alcance la configuración del backend y de la aplicación móvil.

Elementos del entorno

El entorno de desarrollo web requiere Node.js en su versión 18 o superior como runtime, npm como gestor de paquetes incluido con Node.js, y Git para el control de versiones. El proyecto utiliza React 19 con TypeScript, Vite como herramienta de build y desarrollo, Leaflet para la renderización de mapas, TanStack React Query para la gestión de estado del servidor, y Supabase Auth para la autenticación.

Herramientas y Tecnologías involucradas

Node.js proporciona el entorno de ejecución para JavaScript fuera del navegador. npm gestiona las dependencias del proyecto definidas en el archivo package.json. Vite ofrece un servidor de desarrollo con recarga instantánea y un proceso de build optimizado para producción. React 19 con TypeScript permite construir interfaces de usuario tipadas y mantenibles. Leaflet renderiza mapas interactivos con capas GeoJSON para visualizar pórticos y rutas. TanStack React Query maneja las peticiones al backend con caché y

revalidación automática. Supabase Auth proporciona la autenticación de usuarios mediante email y contraseña.

Preparación del ambiente de desarrollo

El primer paso consiste en instalar Node.js descargando el instalador desde el sitio oficial de Node.js. Se recomienda la versión LTS más reciente. Durante la instalación, se debe asegurar que la opción de agregar Node.js al PATH del sistema esté marcada. Para verificar la instalación, se ejecuta en una terminal el comando `node -v`, que debe mostrar la versión instalada, y el comando `npm -v`, que debe mostrar la versión del gestor de paquetes.

Configuración del proyecto

Con Node.js instalado, se procede a clonar el repositorio del proyecto y navegar hasta la carpeta `routes-ui`. Una vez dentro, se ejecuta el comando `npm install` para descargar todas las dependencias definidas en el archivo `package.json`. Este proceso puede tardar algunos minutos la primera vez.

La estructura del proyecto está organizada en carpetas con responsabilidades definidas. La carpeta `api` contiene las configuraciones de axios y las funciones de consulta al backend. La carpeta `app` contiene el contexto de autenticación con Supabase, el layout principal y las páginas de inicio y login. La carpeta `components` contiene los componentes reutilizables como el mapa interactivo, los marcadores de pórticos y la capa de rutas. La carpeta `features` contiene las páginas de administración. La carpeta `hooks` contiene los hooks personalizados para la obtención de datos. La carpeta `types` contiene las definiciones de tipos TypeScript.

Configuración de claves y variables de entorno

Se debe cambiar el host del backend según necesidad, viene por defecto en `localhost:8080`, se encuentra en el fichero `axios.ts`

Verificación de implementación

Para verificar que el entorno está correctamente configurado, se ejecuta el comando `npm run dev` desde la carpeta `routes-ui`. Vite iniciará un servidor de desarrollo y mostrará en la terminal la URL local donde está disponible el panel web, típicamente `http://localhost:5173`.

Se accede a esta URL en el navegador y se verifica que la página de inicio de sesión se muestre correctamente. Se realiza una prueba de autenticación con las credenciales de Supabase para confirmar que las variables de entorno están correctamente configuradas.

Una vez autenticado, se verifica que el mapa interactivo se renderice correctamente y que los datos de pódicos y autopistas se carguen desde el backend. También se verifica la funcionalidad de cálculo de rutas introduciendo coordenadas de origen y destino en el mapa.

Resolución de problemas comunes

Si el comando npm no se reconoce en la terminal, se debe verificar la instalación de Node.js y que esté agregado al PATH del sistema. Cerrar y volver a abrir la terminal suele resolver este problema.

Si npm install falla con errores de permisos, se debe ejecutar la terminal como administrador o utilizar un gestor de versiones de Node.js como nvm para evitar problemas de permisos.

Si el panel web carga pero no muestra datos del backend, se debe verificar el archivo de configuración de axios ubicado en `src/api/axios.ts`. En este archivo se define la URL base del API Gateway que el frontend utiliza para todas las peticiones. Se debe confirmar que esta URL apunte a la dirección correcta donde está corriendo el gateway-service, típicamente `http://localhost:8080`, y que todos los servicios backend estén levantados y funcionando mediante Docker Compose.

Si la autenticación con Supabase falla, se debe verificar la configuración del cliente de Supabase en el archivo `src/app/lib/supabase.ts`. Este archivo inicializa el cliente de Supabase con los datos hardcoded. Si el mapa no se renderiza, se debe verificar la conexión a internet, ya que Leaflet carga los tiles del mapa desde servidores externos. También se debe revisar la consola del navegador en busca de errores relacionados con la inicialización del mapa o la carga de los tiles. En algunos casos, puede ser necesario verificar que el contenedor del mapa tenga una altura definida en el CSS, ya que Leaflet requiere que el contenedor tenga dimensiones explícitas para renderizar correctamente.